Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Files

Introduction to Computer Programming

Management and Artificial Intelligence

LUISS

Persistence

So far...

- we know how to make programs;
- we know how to communicate our intentions to the CPU using conditional execution, functions, and iterations;
- we know that all our variables are stored in the main memory (RAM).

However, recall from past lectures that once your computer is turned off, all RAM and CPU contents are gone.

Specifically, we know that

- the main memory (RAM) is volatile
- ...but our hard disk drive / solid state drive offers permanent storage.

Even more, as soon as the Python interpreter quits, all your variables are gone.

Today let's answer to the following questions:

1. How can we read data from disk?
    - we know how to create variables
    - ...but we cannot always define our data "by hand"
2. How can we store our results permanently?
    - we know how to show variables contents
    - ...but it's not quite customary to screenshot your shell in order to share your results

Focus on:

1. Text files, such as the ones we can create with a simple text editor
2. Pickle files, i.e., files that store Python data

Reading Text Data

Let's start by reading a text file which is stored on our disk.

Python provides a built-in function called `open()` that takes care of opening a file.

```
>>> fID = open("/path/to/file/myFile.txt", "r")
```

The `open()` function takes the following inputs:

1. the path on your disk pointing to the file to be opened (i.e., `myFile.txt`);
2. the `"r"` flag, that means that file has to be opened in *read mode*.

and returns the file *handle* (i.e., `fID`).

Reading Text Data

Let's start by reading a text file which is stored on our disk.

Python provides a built-in function called `open()` that takes care of opening a file.

```
>>> fID = open("/path/to/file/myFile.txt", "r")
```

The `open()` function takes the following inputs:

1. the path on your disk pointing to the file to be opened (i.e., `myFile.txt`);
2. the `"r"` flag, that means that file has to be opened in *read mode*.

and returns the file *handle* (i.e., `fID`).

NOTE: The file handle is not the actual data contained in the file, but instead it is a *handle* that we can use to read the data.

```
f = open('stuff.txt', 'r')
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
Cell In[2], line 1
----> 1 f = open('stuff.txt', 'r')

File ~/.local/lib/python3.12/site-packages/IPython/core/interactiveshell.p
y:324, in _modified_open(file, *args, **kwargs)
    317 if file in {0, 1, 2}:
    318     raise ValueError(
    319         f"IPython won't let you open fd={file} by default "
    320         "as it is likely to crash IPython. If you know what you are
doing, "
    321         "you can use builtins' open."
    322     )
--> 324 return io_open(file, *args, **kwargs)

FileNotFoundError: [Errno 2] No such file or directory: 'stuff.txt'
```

WARNING: If the file does not exist, *open()* throws an error and will not return any file handle. The current Python version has a dedicated error:

```
FileNotFoundError: [Errno 2] No such file or directory: 'stuff.txt'
```

or, in some older Python versions:

```
IOError: [Errno 2] No such file or directory: 'stuff.txt'
```

HINT: Make sure that the file exists in the input path and/or make sure that you have set the input path correctly. The filename (with extension) must be included.

fID.read()

`fID.read([size])` reads `size` characters (in raw format) from the text file:

- it returns a unique string that contains the file contents
- if `size` is omitted, it reads the entire file

```
>>> fID = open("/path/to/file/words.txt", "r")
>>> fileData = fID.read()
>>> print(fileData)
aa\naah\naahed\naahing\naahs\naal\naalii\naaliis\naals\naar ...
dvark\naardvarks\naardwolf\naardwolves\naas\naasvogel\na ...
asvogels\naba\nabaca\nabacas\nabaci\naback\nabacus\nabacu ...
ses\nabaft\nabaka\nabakas\nabalone\nabalones\nabamp\nabam ...
pere\nabamperes\nabamps\nabandon\nabandoned\nabandoning ...
...[truncated output]

>>> print(type(fileData))
<class 'str'>
```

Under the hood, `read()` uses a header in order to keep track of the position within `fID`. If `read()` reaches the end of file `fID`, it returns and empty string. You can't go back (unless you re-open the file).

```
>>> fID = open("/path/to/file/words.txt", "r")
>>> fileData = fID.read()
>>> print(len(fileData))
1016511 # we do have something in fileData

>>> fileDataAgain = fID.read()
>>> print(len(fileDataAgain))
0 # we have an empty string!
```

```
>>> fID = open("/path/to/file/words.txt", "r")
>>> fileData = fID.read(10)
>>> print(fileData)
'aa\naah\naah' # first 10 characters in fID

>>> fileDataAgain = fID.read(10)
>>> print(fileDataAgain)
'ed\naahing\n' # 10 more characters from fID
```

NOTE: In the previous slide, the first call of `fID.read()` scanned the entire file until the end.

The second call sees nothing!

All file objects methods, including the ones we see today:

- `fID.readlines()`
- `fID.readline()`

use an internal header to keep track of the position within the file.

NOTE: In the previous slide, the first call of `fID.read()` scanned the entire file until the end.

The second call sees nothing!

All file objects methods, including the ones we see today:

- `fID.readlines()`
- `fID.readline()`

use an internal header to keep track of the position within the file.

WARNING: `fID.read()` loads the entire content of your file in the main memory.

Use it if at least one of the following holds:

- the text file is small;
- the amount of RAM is sufficient for storing its entire contents.

The same facet also holds for `fID.readlines()`.

fID.readline()

`fID.readline()` reads a single line from the text file.

```
>>> fID = open("/path/to/file/words.txt", "r")
>>> fileData = fID.readline()
>>> print(fileData)
'aa\n' # first line in fID

>>> fileData = fID.readline()
>>> print(fileData)
'aah\n' # second line in fID

>>> fileData = fID.readline()
>>> print(fileData)
'aahed\n' # third line in fID

>>> fileData = fID.readline()
>>> print(fileData)
'aahing\n' # fourth line in fID
```

...and so on

fID.readlines()

`fID.readlines([size])` reads the file contents in such a way that each line is a list item:

- if the optional parameter `size` is given, whole lines totalling approximately `size` bytes (possibly after rounding up to an internal buffer size) are read
- if `size` is not given, it goes up until EOF (end-of-file)

```
>>> fID = open("/path/to/file/words.txt", "r")
>>> fileData = fID.readlines()
>>> print(fileData)
[
    'aa\n',     # first line in fID
    'aah\n',    # second line in fID
    'aahed\n',  # third line in fID
    'aahing\n', # fourth line in fID
    'aahs\n',   # fourth line in fID
    'aal\n',    # fifth line in fID
    ...and so on until...
    'zymurgy\n' # last line in fID
]
```

Iterating Over a File

Finally, you can also read a file, line by line, by iterating over its handle with a very simple `for` loop.

```
>>> fID = open("/path/to/file/words.txt", "r")
>>> for line in fID:
...     print(line)
```

At each loop iteration, a new line from the text file is fetched into `line`. The `for` loop automatically stops when it reaches EOF.

Iterating Over a File

Finally, you can also read a file, line by line, by iterating over its handle with a very simple `for` loop.

```
>>> fID = open("/path/to/file/words.txt", "r")
>>> for line in fID:
...     print(line)
```

At each loop iteration, a new line from the text file is fetched into `line`. The `for` loop automatically stops when it reaches EOF.

NOTE: This is (possibly) the most efficient solution, especially when dealing with large text files since:

- it's fully automatic
- each line is read, processed and then discarded.

Manually closing files vs `with` block

It is customary to close the file after processing:

```
>>> fID = open("/path/to/file/words.txt", "r")
>>> # do something with fID
>>> fID.close()
```

If you do not close the file, Python automatically close it for you when the program ends. If you feel you may forget to close the file, use the `with` statement:

```
>>> with open("/path/to/file/words.txt", "r") as fID:
...     # do something with fID
```

and `fID` is automatically closed at the end of the `with` block.

Manually closing files vs `with` block

It is customary to close the file after processing:

```
>>> fID = open("/path/to/file/words.txt", "r")
>>> # do something with fID
>>> fID.close()
```

If you do not close the file, Python automatically close it for you when the program ends. If you feel you may forget to close the file, use the `with` statement:

```
>>> with open("/path/to/file/words.txt", "r") as fID:
...     # do something with fID
```

and `fID` is automatically closed at the end of the `with` block.

NOTE: The `with` keyword is powerful. It can create context managers for several resources: scheduling threads for parallel processing, reading/writing data, exception handling and others. Yet it's quite an advanced topic.

As you may have noted, every line is treated as a string (regardless of its content).

In order to parse the content of each line, you have to use:

- casting
- regular expressions
- string methods.

As the latter is concerned, notable examples include:

- `s.rstrip([chars])`: returns a copy of a string `s` with trailing characters removed. `[chars]` specifies the set of characters to be removed. It defaults to whitespaces and escape characters (e.g., `\n`)
- `s.lstrip([chars])`: same as above, but it treats leading characters
- `s.strip([chars])`: same as above, but it treats both leading and tailing characters
- `s.split([char])`: splits a string `s` over a given `[char]`. It defaults to whitespaces.

Writing Text Data

In order to write data to a text file you first have to open a file:

```
>>> fID = open("/path/to/file/myFile.txt", "w")
```

The `open()` function takes the following inputs:

1. the path on your disk where you file will be saved (i.e., `myFile.txt`);
2. the `"w"` flag, that means that file has to be opened in *write mode*.

and returns the file handle (i.e., `fID`).

Writing Text Data

In order to write data to a text file you first have to open a file:

```
>>> fID = open("/path/to/file/myFile.txt", "w")
```

The `open()` function takes the following inputs:

1. the path on your disk where you file will be saved (i.e., `myFile.txt`);
2. the `"w"` flag, that means that file has to be opened in *write mode*.

and returns the file handle (i.e., `fID`).

WARNING: If `/path/to/file/myFile.txt` already exists and you open it in write mode, all of its former contents are gone!

Writing some data to a text file is just as easy as doing:

```
>>> fID.write(<some content>)
```

For example:

```
>>> fID.write("This is the first line \n")
>>> fID.write("This is the second line \n")
>>> fID.close()
```

NOTE: `write()` argument must be a string. So make sure you cast your data to a string.

NOTE: You have to close the file especially when you open it in write mode to make sure that all your data are flushed to the disk.

You can also add new contents to an already-existing file by opening it in *append mode* (flag `"a"`):

```
>>> fID = open("/path/to/file/myFile.txt", "a")
```

If you open a file in append mode, any call to the `write()` method will write data to the end of the file.

For example:

```
>>> fID.write("This is a new line \n")
>>> fID.write("This is another new line \n")
>>> fID.close()
```

Pickle Files

Text files are a great option for saving your variables in a human-readable form.

But what happens when you have:

- very complex lists-of-lists (or list-of-tuples)?
- very complex dictionaries?
- instances of a user-defined class?

It can be quite hard to parse these heterogeneous data structures into a text file. Not to mention that in order to load them again, you have also to write a quite complex loader.

In this regard, Pickle files are a great option.

The ***serialisation*** process is a way to convert a data structure into a linear form that can be stored on disk or transmitted over a network.

The Python `pickle` module allows you to serialise objects in a ***binary*** format:

- ✓ the Pickle protocol allows to easily read (*unpickle*) and write (*pickle*) Python data objects with 1 line of code
- ✓ you can serialise the vast majority of Python objects, including:
    - booleans
    - integers, floats, complex numbers
    - strings
    - tuples, lists, dictionaries, sets (if they contain pickable objects)
    - instances of user-defined classes

- ✗ Pickle file contents are not human-readable: you can only read a Pickle file thanks to the `pickle` module in Python
- ✗ Pickle files are only Python-related: cannot be used in other programming languages
- ✗ the Pickle protocol changed across Python versions: unpickling a file that was pickled in a different version of Python may not always work properly

Writing `pickles`

In order to write a Pickle file, you must first `import` the `pickle` module:

```
>>> import pickle
```

Then, choose the object(s) that you want to save, e.g. `L`:

```
>>> L = [ ]
>>> for i in range(3):
...     L.append(i * 2)
```

And then, the magic happens:

```
>>> with open('myfile.pkl', 'wb') as outfile:
>>>     pickle.dump(L, outfile)
```

What's going on in the last code snippet?

1. we `open()` a file named `myfile.pkl` in `wb` mode (*write binary*) and let `outfile` be its file handle
2. we write `L` to the file
3. the context manager from `with` closes the file for us

You can also save multiple objects by "packing" them into a list or tuple, e.g.:

```
>>> with open('myfile2.pkl', 'wb') as outfile:
>>>     pickle.dump([L1, L2, s1, n1], outfile)
```

You can also save multiple objects by "packing" them into a list or tuple, e.g.:

```
>>> with open('myfile2.pkl', 'wb') as outfile:
>>>     pickle.dump([L1, L2, s1, n1], outfile)
```

WARNING: When you read back the pickle file with a list/tuple that contains multiple items, you must remember their order!

You can also save multiple objects by "packing" them into a list or tuple, e.g.:

```
>>> with open('myfile2.pkl', 'wb') as outfile:
>>>     pickle.dump([L1, L2, s1, n1], outfile)
```

WARNING: When you read back the pickle file with a list/tuple that contains multiple items, you must remember their order!

NOTE: The `wb` flag is mandatory to tell Python to write a (serialised) binary file. Do not confuse `w` with `wb`!

You can also save multiple objects by "packing" them into a list or tuple, e.g.:

```
>>> with open('myfile2.pkl', 'wb') as outfile:
>>>     pickle.dump([L1, L2, s1, n1], outfile)
```

WARNING: When you read back the pickle file with a list/tuple that contains multiple items, you must remember their order!

NOTE: The `wb` flag is mandatory to tell Python to write a (serialised) binary file. Do not confuse `w` with `wb`!

NOTE: The file can be saved also without extension (i.e., `myfile` instead of `myfile.pkl`) and in principle you can also use the extension you prefer (i.e., `myfile.pickle` instead of `myfile.pkl`): Python will still understand that's a Pickle file.

Reading `pickles`

In order to read a Pickle file, you must first `import` the `pickle` module:

```
>>> import pickle
```

and then, the magic happens:

```
>>> with open('myfile.pkl', 'rb') as infile:
>>>     X = pickle.load(infile)
```

What's going on in the last code snippet?

1. we `open()` a file named `myfile.pkl` in `rb` mode (*read binary*) and let `infile` be its file handle
2. we read the contents of `infile` into variable `X`
3. the context manager from `with` closes the file for us

Compressing Pickle Files

If you have to save large data structures, Pickle files may take a lot of space on your hard drive.

Compressing Pickle Files

If you have to save large data structures, Pickle files may take a lot of space on your hard drive.

Workaround? **Compression**.

Compression is the art of encoding (hence, saving) data with fewer bits in order to save disk space.

Compressing Pickle Files

If you have to save large data structures, Pickle files may take a lot of space on your hard drive.

Workaround? **Compression**.

Compression is the art of encoding (hence, saving) data with fewer bits in order to save disk space.

WARNING: Do not confuse serialisation with compression!

Compressing Pickle Files

If you have to save large data structures, Pickle files may take a lot of space on your hard drive.

Workaround? **Compression**.

Compression is the art of encoding (hence, saving) data with fewer bits in order to save disk space.

WARNING: Do not confuse serialisation with compression!

Since there is no free lunch, compression is driven by a trade-off:

- saves disk space by encoding data with fewer bits
- the encoding stage can be costly (i.e., in terms of running times)

Python in its standard library provides two well-known compression algorithms:

- *gzip*, in the `gzip` module
- *bzip2*, in the `bz2` module

Again, since there is no free lunch, compression algorithms themselves do have some trade-offs:

- *bzip2* is slower than *gzip*
- *gzip* yields files with larger size

For the sake of example, let us consider *bzip2* (although similar interfaces hold for *gzip*).

Writing a Compressed Pickle File

In order to write a compressed Pickle file with *bzip2*, you must `import` both modules:

```
>>> import pickle
>>> import bz2
```

Let's consider a big list:

```
>>> with open("words.txt", "r") as fID:
>>>     fileData = fID.readlines() # fileData is a list with 113783 words
```

Now, let's save the list with `pickle`:

```
>>> pickle.dump(fileData, open("words.pkl", "wb"))
```

Now, let's save the list with a `bz2`-compressed `pickle`:

```
>>> with bz2.BZ2File("words.bz2", "w") as bz2file:
>>>     pickle.dump(fileData, bz2file)
```

What's going on in the last code snippet?

1. from the `bz2` module, we call `BZ2File` to create a file called `words.bz2` in *write mode* (flag `w`). This yields a file handle named `bz2file`.
2. we use the already-known `pickle.dump()` to save our variable `fileData`. However, instead of passing as argument a file handle obtained via `open()`, we pass the file handle obtained via `bz2.BZ2File()`
3. the context manager from `with` closes the `bzip2` file for us

Result?

- `words.pkl` takes 1.2MB (=1200KB) on disk
- `words.bz2` takes 403KB on disk

The `bz2` file takes 1/3rd of the space with respect to the original `pkl` file.

Reading a Compressed Pickle File

In order to read a compressed Pickle file with *bzip2*, you must `import` both modules:

```
>>> import pickle
>>> import bz2
```

and then:

```
>>> with bz2.BZ2File("words.bz2", "r") as bz2file:
>>>     X = pickle.load(bz2file)
```

What's going on in the last code snippet?

1. from the `bz2` module, we call `BZ2File` to read a file called `words.bz2` in *read mode* (flag `r`). This yields a file handle named `bz2file`.
2. we use the already-known `pickle.load()` to import the contents of `bz2file` into a variable `X`. However, instead of passing as argument a file handle obtained via `open()`, we pass the file handle obtained via `bz2.BZ2File()`
3. the context manager from `with` closes the `bzip2` file for us